Customer Sentiment Analysis Using Natural Language Processing and K-Nearest Neighbors

Abstract

Understanding customer sentiment is critical for organizations seeking to improve products and services based on user feedback. This project implements a Natural Language Processing (NLP) pipeline to classify customer product reviews as positive or negative using a manually implemented K-Nearest Neighbors (KNN) classifier.

The pipeline includes text preprocessing, TF-IDF feature extraction with n-grams, vector normalization, hyperparameter tuning through cross-validation, and final prediction generation. The model was evaluated using accuracy and error rate across multiple values of K to determine the optimal configuration.

The final model achieved a leaderboard score of 0.77. This demonstrates that a carefully engineered classical NLP pipeline combined with a simple distance-based classifier can perform competitively on sentiment classification tasks.

Introduction

Customer reviews provide valuable insights into consumer perceptions of products and services. However, the sheer volume of online feedback makes manual analysis impractical. Sentiment analysis, a subfield of Natural Language Processing (NLP), addresses this challenge by automatically classifying textual data according to sentiment polarity.

The objective of this project is to develop a sentiment classification model capable of identifying whether a customer review expresses positive or negative sentiment. The approach leverages a TF-IDF text representation combined with a custom implementation of the K-Nearest Neighbors algorithm.

Unlike many sentiment analysis implementations that rely on prebuilt machine learning models, this project emphasizes building the classifier from first principles, providing deeper insight into the mechanics of similarity-based learning algorithms.

Dataset Description

The dataset consists of customer product reviews, each labeled with a sentiment value:

- +1 → Positive review
- -1 → Negative review

Each record contains:

Field	Description
Sentiment Label	Binary sentiment classification
Review Text	Customer written product review

During the initial data inspection, several data quality issues were identified:

- Formatting inconsistencies in sentiment labels (+1, -1)
- A small number of missing review entries
- Irregular spacing in certain records

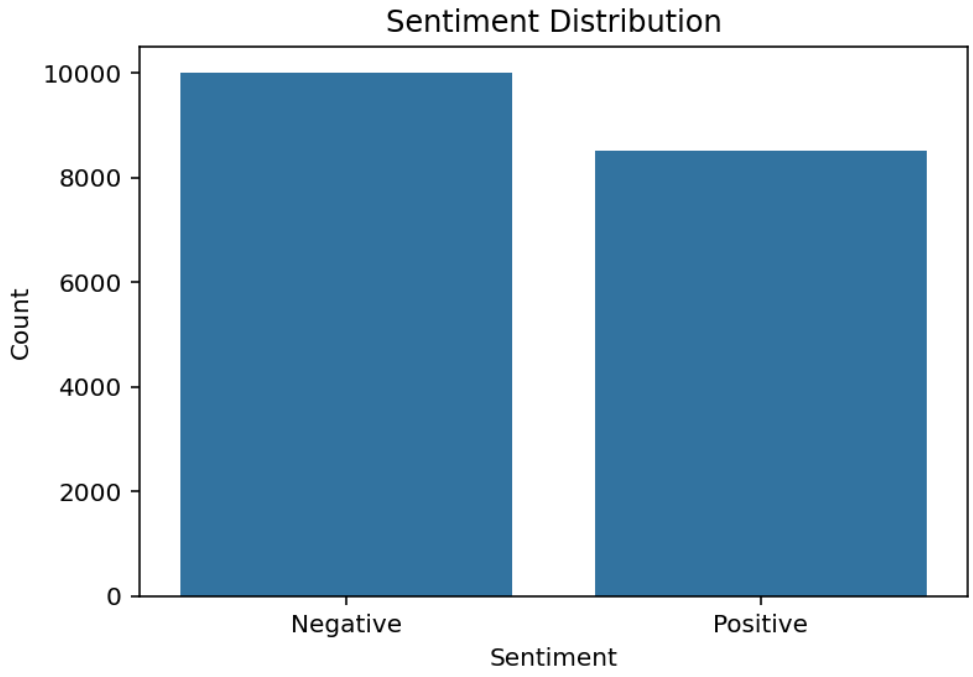
Rows containing null values were removed to maintain dataset integrity. After preprocessing, the dataset contained 18,506 valid reviews used for model training and evaluation.

Exploratory Data Analysis (EDA)

Before modeling, exploratory analysis was conducted to better understand the textual data.

Sentiment Distribution

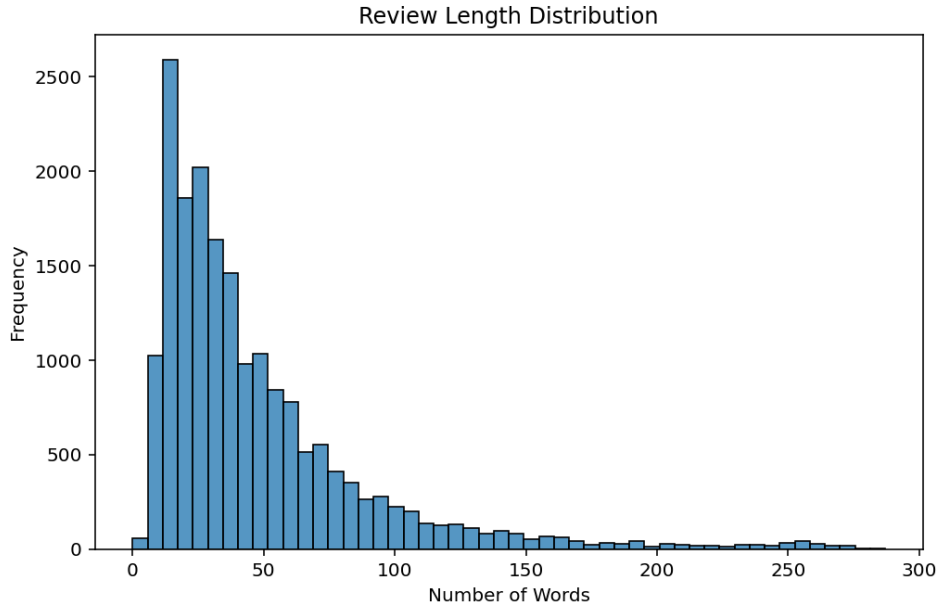
The dataset was inspected to determine whether the classes were balanced. Balanced datasets typically lead to more stable classifier performance.



A bar chart showing the count of positive vs negative reviews.

Review Length Distribution

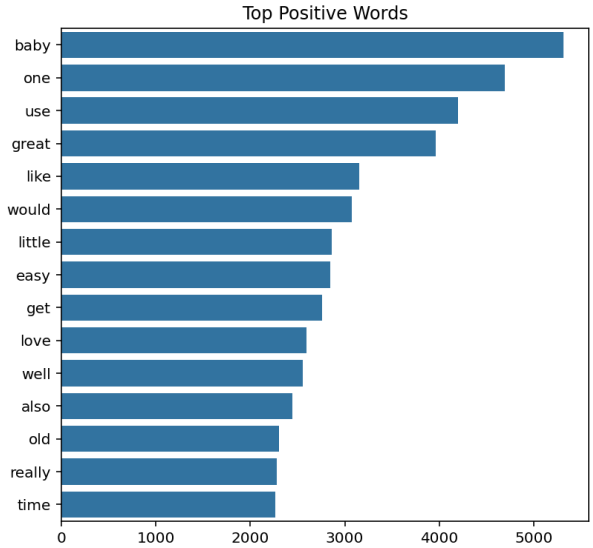
Review length analysis provides insights into the variability of customer feedback.

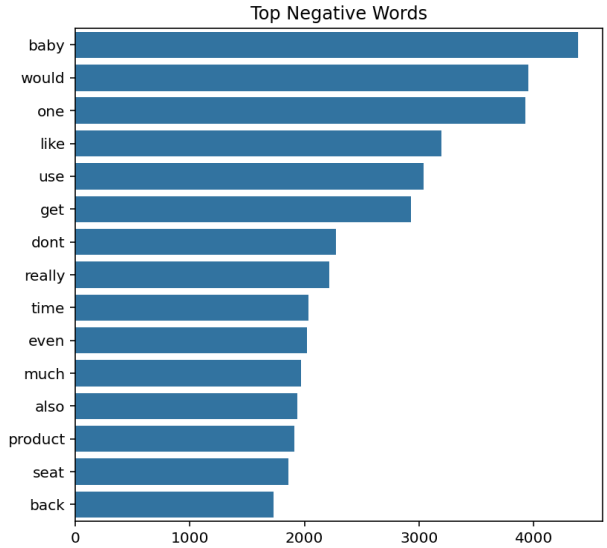


Histogram showing number of words per review.

Word Frequency Analysis

Word frequency analysis helps identify the most common terms appearing in customer feedback.





- Top words in positive reviews
- Top words in negative reviews

These visualizations reveal language patterns associated with sentiment.

Patterns observed:

Positive Terms	**Negative Terms**
baby	baby
one	would
use	one
great	like

These patterns confirm that the dataset contains clear sentiment signals suitable for classification.

Word Cloud

Word Cloud

Text Preprocessing

Raw textual data must be cleaned and standardized before it can be converted into numerical features.

A preprocessing function was implemented to perform the following steps:

1. Convert all text to lowercase
2. Remove punctuation
3. Remove numeric characters
4. Remove stopwords (common words such as the, and, is)
5. Apply stemming to normalize word variants

Example transformation:

Original Review	Processed Review
"This book is such a life saver."	book life saver

The cleaned text provides a more informative representation for feature extraction and reduces noise in the dataset.

Feature Engineering: TF-IDF Representation

After preprocessing, reviews were transformed into numerical feature vectors using Term Frequency–Inverse Document Frequency (TF-IDF).TF-IDF assigns higher weights to words that are important within a document but uncommon across the entire corpus.

Key enhancements implemented in the refactored pipeline include:
N-gram Features
Both unigrams and bigrams were included to capture contextual information.

Examples:

Unigram	Bigram
great	works great
waste	waste money

This allows the model to capture meaningful phrases that single words may not convey.

Feature Dimensionality

The TF-IDF vectorization produced approximately 38,646 unique vocabulary features across the training corpus. To ensure compatibility between training and test datasets, the vectorizer was fit on the training data and reused to transform the test data.

Vector Normalization

TF-IDF vectors were normalized to ensure that cosine similarity calculations reflect relative document similarity rather than absolute magnitude.

K-Nearest Neighbors Classifier

A custom K-Nearest Neighbors (KNN) classifier was implemented to perform sentiment classification. KNN is a distance-based learning algorithm that classifies new data points based on the labels of their nearest neighbors in feature space.

Distance Metric

The classifier uses cosine similarity to measure similarity between TF-IDF vectors:

$$Similarity = \frac{A \bullet B}{|A|\ |B|}$$

Cosine similarity is particularly well suited for high-dimensional sparse text representations.

Classification Procedure

For each review in the test set:

1. Compute cosine similarity between the review and all training reviews
2. Identify the K most similar reviews
3. Retrieve the sentiment labels of those neighbors
4. Assign the majority label as the prediction

This process is repeated for each review in the dataset.

Hyperparameter Tuning

The primary hyperparameter for the KNN algorithm is the number of neighbors (K).
To determine the optimal value of K, cross-validation was performed.

Candidate K Values

Odd numbers between 3 and 45 were evaluated:
K = 3, 5, 7, ..., 45

Odd values were selected to avoid ties in the majority voting step.

Evaluation Metrics

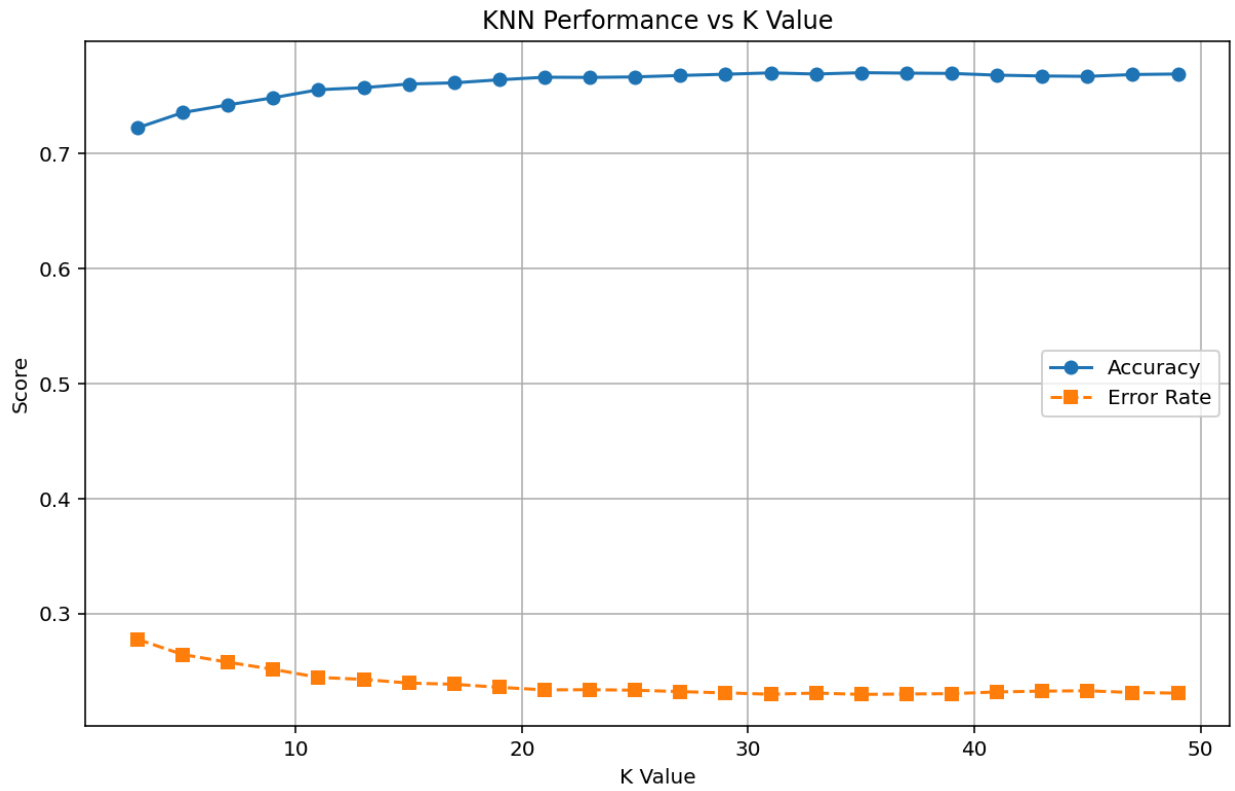
Two performance metrics were tracked:

- Accuracy
- Error Rate

Model Selection

The optimal K value was selected based on:

- Highest accuracy
- Lowest error rate



The analysis showed that K = 35 provided the best performance.

Model Evaluation

After selecting the optimal K value, the final classifier was trained on the full training dataset and used to generate predictions for the test set.

Performance Summary:

Metric	**Value**
Optimal K	35
Accuracy	~0.77

This result indicates strong performance given the simplicity of the model and the constraint of implementing KNN manually.

Key Takeaway

The results demonstrate that classical NLP techniques combined with a simple distance-based classifier can achieve competitive performance on sentiment classification tasks.

Several factors contributed to the model's effectiveness:

1. Comprehensive text preprocessing
2. TF-IDF feature weighting

3. Inclusion of n-gram features
4. Cosine similarity distance metric
5. Systematic hyperparameter tuning

While more advanced models such as neural networks or transformer-based architectures may achieve higher accuracy, the approach used in this project emphasizes interpretability, simplicity, and full algorithmic transparency.

Despite strong performance, several limitations remain:

- KNN is computationally expensive for large datasets since distance must be computed for every training instance.
- TF-IDF representations cannot capture deeper semantic relationships between words.
- The model does not leverage contextual embeddings such as those used in modern language models.

Future Improvements

Several improvements could further enhance the model:

1. Word Embeddings
    a. Using embeddings such as Word2Vec, GloVe, or BERT would allow the model to capture semantic meaning beyond simple word frequency.
2. Dimensionality Reduction
    a. Techniques such as Truncated SVD (LSA) could reduce TF-IDF dimensionality while preserving important semantic structure.
3. Alternative Classifiers
    a. More advanced classifiers could improve performance:
        i. Logistic Regression
        ii. Support Vector Machines
        iii. Gradient Boosting
        iv. Deep Neural Networks
4. Approximate Nearest Neighbor Search
    a. For large datasets, algorithms such as FAISS or KD-Trees could accelerate KNN search.